

My contributions to the TML4 pipeline

A running account of my work on a tool that trains a model on a gene-expression dataset you upload, then predicts, per patient, whether a treatment is likely to work. **My work covers two of the project's branches:** `as`, an independent implementation I built from scratch, and `merge/as`, the team's merged product, which I took over, hardened, and have been running the real experiments on since.

ANGELINA	BRANCHES AS · MERGE/AS	27 MAY – 17 AUGUST 2026		
7	11	275	5	9
PIPELINE MODULES	CLASSIFIERS MAINTAINED	SETUPS PER FULL GRID	REAL COHORTS	CONCLUSIONS WITHDRAWN

In one paragraph

I built a complete pipeline of my own on the `as` branch — pluggable models and feature maps behind registries, a full clinical metric report, and machinery that tells a user whether the model it just handed them is actually better than chance on their data. I then used it to run the project's real science: eight benchmark datasets and three treatment-response cohorts, ending in a ten-day investigation that reproduced a published biomarker, showed it was almost certainly circular, and proved it on an independent 298-patient cohort. Since July I have owned the merged product on `merge/as`: finding and fixing the classifier bugs that were corrupting the team's benchmark numbers, making the exhaustive search fast enough to be practical, closing a live patient-data leak, and redesigning the interface so a model that did not learn can never be displayed as though it did. Since mid-August I have been using it as an instrument: full 275-setup grids on a real 298-patient cohort, an imaging comparison, and a cross-cohort test that made me correct my own statistics.

My role on the team

The project began as five independent implementations of the same idea, on separate branches. Mine was `as`: I was not merging or coordinating, I was building a second complete answer to the problem — and, more usefully, building the apparatus to find out whether any of our answers were real. Most of what I contributed to the team’s thinking came out of that apparatus rather than out of the pipeline itself: the permutation test, the nested-CV honesty check, the majority-baseline reporting, and the benchmark that put our methods and the published ones on the same footing.

When the branches were reconciled into one seven-module pipeline, I took over `merge/as` and my role shifted to the core of that merged product: **correctness, performance, honesty guarantees, interface and security**. Concretely, that means the classifier and cross-validation bugs, the parallel scheduler, the feature maps ported across from my own branch, the explainability module, the guard that refuses to report a non-learning model as a real result, and the security work before any public launch. My teammate Sulaf owns the integration itself, the cluster profiling harness, and the deployment research; several of the fixes below started as findings in her profiling runs, and several of my findings went back the other way.

What went into the team’s code

The work below left my branch and changed shared behaviour — either by fixing something every teammate’s technique ran through, or by correcting numbers the team had already published.

CORRECTNESS · BENCHMARK TABLE

Unstratified cross-validation on two custom classifiers

A base-class ordering bug meant two of our estimators were scored unstratified while every scikit-learn model got stratified folds. **Corrected the team’s published numbers on five datasets**; both classifiers had been underrated.

CORRECTNESS · ALL 8 DATASETS

The RBF kernel collapse

Reported by Sulaf’s profiling as a scikit-learn default problem; the cause was our own hardcoded kernel width, ~800× too large for 16,000-gene data. **All eight datasets went from collapsed to learning**.

CORRECTNESS · SMALL COHORTS

Adaptive PCA and adaptive boosting leaf size

Answers Sulaf’s finding that PCA failed outright on small cohorts. Components and leaf size now clamp to the data instead of erroring or refusing to split. **Recovered ~40 configurations per small dataset**; identical wherever data was already large enough.

CORRECTNESS · NAIVE BAYES

Variance smoothing on near-zero-variance components

Localized the residual collapses to one feature map by sweeping all preprocessor × map pairs. **32 of 32 combinations now learn, down from 11 collapses**.

PERFORMANCE · SHARED HARNESS

A 3.4-hour configuration, and a redundant evaluation pass

An unbounded solver iteration limit made one grid cell take 12,115 seconds; and the evaluator was refitting every fold twice. **3.4 hours → 1.22 seconds, and a 1.6× speedup on every evaluation** — including Sulaf’s cluster runs.

PERFORMANCE · SHARED MODULE

Parallel scheduling, extracted so both callers share it

The app had the longest-job-first speedup and the cluster harness did not. Moved into the pipeline package so there is one implementation, not two that drift.

REPORTING · EVERY RESULT

The honesty guard

A model that predicts one class for everyone reports its accuracy as the class balance. The guard flags it, excludes it from the automatic pick, and states plainly that it did not learn — then extended to non-converged fits.

SECURITY · LIVE DEPLOYMENT

Cross-session patient-data leak

Reports were written to a shared path in a single-process server, so one visitor could download another’s per-patient predictions. Confirmed by hash, then closed. Also replaced the pickle model format Sulaf flagged with an authenticated-encrypted one.

CAPABILITY · MERGED PIPELINE

Feature maps, explainability, imaging

Ported log2 and supervised gene selection from my branch after measuring that they help; built the gene-level explanation module; integrated a teammate’s imaging module without letting its older code undo the leak fix. **The size guard moved to the one function every data source converges on**, so no future source can arrive unbounded.

Contents

The rest of this document is the detail, in three parts.

Part I Build it, then try to break your own result

[as](#)

- Tuning the instrument, not the result
- Registries, so a method costs one file
- The first comparison, and the first retraction
- The default model was chosen on the wrong data
- What robustness looks like in a table
- Two findings I would now call foreshadowing

Part II The IMPRES investigation

[as](#)

- Is any of this real?
- A fair critique, tested rather than argued
- Searching harder makes things worse
- Fixing a benchmark that flattered me
- “No signal” was wrong — the method was
- Arguing against my own best result
- Building the instrument
- The verdict, and three results I did not want

Part III Make it a product, and keep it honest

[merge/as](#)

- Explainability through PCA
- Four bugs, one theme
- A speedup, and the bug inside it
- The hole I found in my own guard
- Reading the framework’s source
- The manual, and what it got wrong
- Design as an extension of the argument
- The leak that was already live
- Where a secret could escape
- Integrating a branch without losing fixes
- Naming the thing correctly
- Imaging: measured, not argued

Part IV Measuring what we shipped

[merge/as](#)

- Shipping a real cohort
- The first full grid on real patients
- One column against 26,128 genes
- Threshold tuning, measured and rejected
- The tile cohort I built wrong
- 3-D against 2-D, same patients
- The luck line
- The right measurement, wrong data
- Re-running after the fix

Build it, then try to break your own result

27 MAY – 6 JULY 2026

Tuning the instrument, not the result

The first entry in my log is not a result, it is a calibration. With an effect size of 0.8 the synthetic data was too easy — linear SVM hit 93.75% validation accuracy, and three models all pinned near the ceiling would have told me nothing about which was better. I swept it down to 0.65, landing SVM at 77.50%, so the algorithms had room to differ.

That is the shape of everything after it: change the measuring apparatus deliberately and say so, rather than reporting whichever number came out first.

Registries, so adding a method costs one file

Models and feature maps each live behind a registry — a dict mapping a key to a module with a fixed interface. Adding a fourth model means one new file and one dict entry, with no call-site changes anywhere. The CLI's `--model` choices, the GUI dropdown, and later the automatic configuration search all read the same registry, so a new method appears in all three for free.

This paid off repeatedly. When I added supervised feature selection, maps needed labels at fit time, so I widened the interface to `fit(X, y=None)` — backward compatible, five existing maps updated to ignore `y`, three call sites passing labels. Eight feature maps and four models later, nothing about that decision has needed revisiting.

The first comparison, and the first retraction

Three models on synthetic data: Gradient Boosting 80.00% validation, SVM 77.50%, Random Forest 73.75%. I wrote up *why*, which turned out to matter more than the ranking. The planted signal is additive across 30 informative genes out of 3000 — 1% signal density — close to exactly what stagewise boosting is built for. And Random Forest's per-split feature subsampling defaults to $\sqrt{3000} \approx 55$ features, so the expected number of informative features visible at any single split is $55 \times 30/3000 \approx 0.55$. More than half of all splits see *zero* informative features and are forced to split on noise. Bagging averages over many such trees but cannot manufacture signal the trees never saw.

Then I ran it across five seeds, and had to retract three specific claims from my own analysis.

SUPERSEDED · THREE CLAIMS, BY NAME

SVM has the best validation AUC. Random Forest has a dangerously low 52.6% test sensitivity. SVM has the largest validation-to-test AUC drop.

All three were single-seed artifacts. Across five seeds, boosting leads on mean validation AUC (0.899 vs 0.840) and wins 4 of 5; Random Forest's sensitivity is the *most variable* rather than the lowest, ranging 0.526–0.905; and in 3 of 5 seeds at least one model's test AUC *exceeded* its validation AUC. **What survived:** boosting best on average, Random Forest worst overall.

The default model was chosen on the wrong data

Gradient Boosting was the default because it won on synthetic data. Then I ran the Tan benchmark — eight real cancer microarray datasets, five seeds each, only the split seed varying since real data cannot be regenerated.

CROSS-DATASET MEAN ACCURACY · 8 DATASETS × 5 SEEDS

MODEL	MEAN ACCURACY
Support Vector Machine	0.9045
Random Forest	0.8676
Gradient Boosting	0.8368

SUPERSEDED

Gradient Boosting is the right default.

It was calibrated to a regime that does not transfer. Real data lives at $p \gg n$ — 7,000–16,000 genes against 33–280 patients — where linear SVM with L2 regularisation is the better first choice. **Default switched to SVM** across the GUI, the CLI and the docs.

The same sweep killed something smaller and more tempting. Single-seed, SVM had scored a perfect 1.000 on four of the eight datasets. Multi-seed: 0.975, 0.947, 0.989, 0.971. All excellent, none actually perfect. Only one was stable enough (std 0.013) to be worth quoting.

What robustness looks like in a table

I added k-Top Scoring Pairs as an exploratory fourth model — a rank-based classifier that votes on “is gene i below gene j ” within fixed pairs, making it invariant to any per-sample rescaling. It came second of four on the real benchmark (0.8896, ahead of both Random Forest and boosting) and last on synthetic, which is exactly right: the synthetic signal is additive in magnitude and this model discards magnitude by design.

The more interesting finding is where its robustness shows up. Not in the means — in the variance. On one dataset its standard deviation across seeds is 0.027 against SVM's 0.050; on another, 0.030 against 0.076. Rank-invariance buys per-seed stability rather than headline accuracy, which is the property this family of biomarkers is actually claimed for.

I then made that property available to every model as a front-end, replacing each patient's vector with within-patient ranks. Under a per-sample rescale $x' = a \cdot x + b$ — the kind of normalization difference you get between cohorts:

SVM ACCURACY UNDER A PER-SAMPLE SHIFT

FRONT-END	NO SHIFT	WITH SHIFT
identity (raw magnitude)	0.8075	0.6050
rank transform	0.7725	0.7725

Identical to four decimal places, because within-row ranks cannot change under a per-sample monotone transform. The cost is about four points of accuracy when there is no shift. That is the whole robustness-versus-accuracy trade, made concrete rather than argued. I logged the limit too: this is a *per-sample* shift, which ranking fixes exactly; arbitrary *per-gene* batch effects are not removed by ranking, and I did not claim they were.

Two findings I would now call foreshadowing

A feature-map ablation showed PCA at 100 components underperforming everywhere. The reason was not PCA — it was the clamp, $\min(100, n_{\text{train}} - 1)$, which on the small datasets meant 18 components on one and 36 on another. A default silently mismatched to the data. I would meet that same failure mode four more times on the other branch.

And cross-cohort validation was blocked outright: one cohort labelled genes with symbols, the other with Entrez IDs, giving **zero** shared identifiers. I wrote a general harmonizer over NCBI's gene table — official symbols win, ambiguous synonyms dropped — mapping 95.2% and 100% of the two cohorts into a common space for **21,488 shared genes where exact matching gave 0**.

The IMPRES investigation

5 – 14 JULY 2026

One continuous argument that took ten days, changed direction four times, and ended by demolishing its own best result. It is the work I would put forward first.

Is any of this real?

The two treatment-response cohorts I had were small — 27 and 33 patients. Accuracies hovered near chance and I could not tell whether that was weak signal or no signal. So I built a permutation test: shuffle the labels, re-run the entire cross-validated evaluation, repeat a hundred times, and compare the real score against that null distribution.

PERMUTATION SIGNIFICANCE · 100 SHUFFLES

DATA	OBSERVED AUC	NULL (SHUFFLED)	P
Synthetic (planted signal)	0.899	0.501 ± 0.041	0.020
Cohort A (n = 27)	0.525	0.516 ± 0.108	0.485
Cohort B (n = 33)	0.617	0.478 ± 0.118	0.109

Cohort B’s 0.617 had looked promising for weeks. It sits inside the null. And the number that explains the entire project is in the third column: at $n \approx 30$ the shuffled-label null has a spread of ± 0.11 , which is why every method I had tried was unstable.

A fair critique, tested rather than argued

I had been feeding raw expression values to the models. Published methods log-transform first. That is a real methodological criticism, so I added $\log_2(x+1)$ as a feature map and re-ran.

REVISED

The near-chance results are the data, full stop.

Log2 lifts Cohort A’s SVM from 0.525 to 0.631 — we genuinely were under-preprocessing. But 0.631 still gives $p = 0.134$. **Better methodology moves the point estimate; significance still needs n.**

Searching harder makes things worse, and I can prove it

The product’s premise is that the framework picks a good configuration for you. So I built that: rank all 24 feature-map × model combinations by out-of-fold AUC, take the winner. Then I built the check on it — nested cross-validation, where an inner loop picks the configuration and an outer loop scores the whole procedure.

OPTIMISTIC SEARCH VS. NESTED CROSS-VALIDATION

DATA	BEST OF 24	NESTED CV	STABILITY OF THE PICK
Synthetic	—	0.839	same config, every fold
Cohort A (n = 27)	0.756	0.333	4 configs across 5 folds
Cohort B (n = 33)	0.620	0.235	5 configs across 5 folds

On real data the optimistic number is almost entirely selection bias — searching 24 configurations on 30 patients finds noise that does not replicate, and the collapse is to *below* chance. Crucially, nested CV does not collapse the synthetic result, so this is an honesty check rather than blanket pessimism. The instability itself turned out to be diagnostic: a different configuration winning nearly every fold is the signature of noise-chasing. Reporting the optimistic winner to a user at this sample size would have been badly misleading.

Fixing a benchmark that flattered me

Then I found my own benchmark was unfair — and unfair in *my* favour. The SVM row got log2; the published immune signatures were being scored on raw counts. I normalized once and fed the same matrix to everything.

The check that made me trust the fix: SVM (0.570) and the pairwise method (0.726) are provably unchanged, because log-then-SVM is identical and a rank-based method is log-invariant. Only the previously unfair rows moved, each up 0.02–0.03. If anything else had moved, the fix would have been wrong.

“No signal” was wrong — the method was wrong

REVISED

Neither cohort carries usable signal.

On a fair benchmark, biology-driven methods beat my generic ML on Cohort B and reach the honest published range: a pairwise rule over checkpoint genes scores 0.726 ($p = 0.069$) against my SVM’s 0.570. **The cohort was not signal-free. My method was wrong for it.**

So I implemented the real thing. IMPRES is a published rule: 15 fixed pairs of immune checkpoint genes, scored by counting how many pairs have gene 1 above gene 2. I transcribed the pairs from Carter et al.’s 2019 reproduction and cross-checked them against the features named in the original Auslander 2018 text.

COHORT	METHOD	AUC	P
A (n = 27)	IMPRES, literal	0.825	0.010
A	SVM + log2 (mine)	0.631	0.139
A	same pairs, re-learned here	0.500	0.604
B (n = 33)	IMPRES, literal	0.752	0.040
B	SVM + log2 (mine)	0.570	0.238

The first method significant on either cohort, and it cleared both. Cohort A's 0.825 matches the published ~ 0.83 , so the implementation is faithful. And the published pairs beat pairs re-learned on the same data by 0.825 to 0.500 — for a robust pair signature, use the published pairs rather than relearning them on thirty patients.

Arguing against my own best result

This was the strongest number in the project, and I wrote the case against it into the log the same day, before running anything that could settle it: both cohorts were plausibly inside IMPRES's original derivation, and Carter et al.'s central critique is precisely that the selection saw the validation data. My permutation test shuffles labels — it cannot detect selection bias introduced *outside* my pipeline. I recorded it as “not yet proof of a real generalizable predictor”: a reproducible published baseline that needed testing on a cohort outside the derivation.

Which meant I had to go and get one.

Building the instrument

The cohort that would settle it — 298 patients, a different tumour type, outside the derivation — is distributed as an R package. I did not want an R runtime in this project, so I wrote a one-time extractor that reads the R serialization directly in Python via the `rdata` parser, monkeypatched to treat unhandled promise nodes as `None`, pulling the count matrix and the response field into a compact archive. The parser is an extraction-only dependency, deliberately kept out of `requirements.txt` — the loader itself needs only numpy.

The verdict

OVERTURNED

Literal IMPRES is a strong, significant predictor (0.825 / 0.752).

On the independent cohort it collapses to 0.560, $p = 0.078$ — not significant, and the *worst* method in the table. The melanoma performance was in-sample optimism. **Carter et al.'s non-reproducibility critique, reproduced on fully independent data.**

The same run vindicated the thing I had been claiming all along. Five methods cleared $p < 0.05$, led by my own SVM + log2 at 0.697 ($p = 0.020$) — the first honest significant result in the project. Nothing changed except n : 298 instead of 30. The bottleneck was statistical power, not the classifier and not the biology.

SUPERSEDED

Biology-driven methods beat generic machine learning.

Also a small-sample artifact. The flexible learner that was worst at $n \approx 30$ is best at $n = 298$.

Which method wins is a function of how much data you have — the empirical case for letting the framework choose per dataset, and for checking that choice honestly.

Three more results I did not want

Models do not transfer between cohorts. I harmonized three cohorts into a shared 21,486-gene space and held each out in turn. Every held-out AUC lands at ≈ 0.50 , whether the training cohorts share a tumour type or not. Uncomfortable, but it validates the product's design directly: the app trains on the cohort *you* upload, and it should never imply a model carries over to someone else's data.

The published headroom is mostly optimism. Honest supervised feature selection on the large cohort reaches nested-CV 0.671 ($p = 0.024$) — real signal, and *below* the fixed SVM + log2 at 0.697. The optimistic search says 0.737. So the gap between my results and the ~ 0.75 in the literature is largely the selection bias that honest evaluation removes, not reproducible signal I was failing to capture. Getting that verdict at all needed a performance path: a label-free variance pre-filter plus parallel permutations brought a previously intractable run down to 449 seconds on a 31,000-gene cohort.

And the most uncomfortable one. Tumour mutational burden — a single number, not an expression model at all — scores 0.715 ($p = 0.005$) on the same samples, beating my whole expression pipeline. Signatures plus that one number (0.719) is indistinguishable from it alone, so expression is redundant given it. The real accuracy lever here is a different data modality, not a better model of mine. I logged it as the headline finding rather than a footnote, because a log that only records the flattering results is not worth keeping.

Make it a product, and keep it honest

1 JULY – 10 AUGUST 2026

Explainability, and the part that was supposed to be hard

The pipeline’s seventh module answers “which genes pushed this patient’s prediction?” The hard version of that question is what happens when the feature map is PCA: the model sees components, not genes, and inverting the projection to get back to gene space is the usual sticking point.

I sidestepped it. For one patient, occlude each raw gene toward the training-set mean and re-run the *entire fitted machine*, measuring how the predicted probability of responding moves. Because it runs the whole pipeline rather than inspecting the model, it is model-agnostic across all eleven classifiers and returns gene-level attributions even when PCA sits in the middle — solved by construction rather than by inversion. A candidate pre-filter keeps it fast on wide matrices.

Four bugs, one theme

A single configuration was taking 3.4 hours. On a grid run, one preprocessing-and-classifier combination took 12,115 seconds. With an unbounded iteration limit, the SVM solver grinds indefinitely on badly conditioned input. I capped it: that configuration now finishes in 1.22 seconds, and healthy configurations converge at 545 and 715 iterations — orders of magnitude below the cap — so their models are provably bit-identical. Same session: logistic regression needed 6,445 iterations against a 5,000 cap, so it had been silently truncating and flooding the logs with warnings.

The RBF kernel was collapsing on 98% of runs. Sulaf’s profiling run found the RBF SVM predicting a single class in 126 of 128 combinations across every dataset, and attributed it to a scikit-learn default. Reading the code, the cause was ours: a hardcoded kernel width roughly 800× too large for 16,000-gene data, which collapses the kernel matrix toward the identity so the model learns nothing. One line, and all eight datasets went from collapsed to learning.

Naive Bayes was collapsing on eleven. I swept all four preprocessors × four feature maps to localize it: every one was class-conditional PCA, whose trailing components have near-zero variance, letting the variance-smoothing floor blow up so a few features dominate the likelihood. Raising it gave 32 of 32 combinations learning across all eight datasets — two better, eleven unchanged, two slightly worse.

And one that had been quietly corrupting the team’s benchmark table. My verification of a new estimator failed on larger data — one dataset dropped 0.948 to 0.753 — but direct fits were identical, so the problem was in cross-validation. Every custom estimator in the project declared

`(BaseEstimator, ClassifierMixin)`. Under scikit-learn ≥ 1.6 , tag resolution follows the method resolution order, so `BaseEstimator` wins, `is_classifier()` returns `False`, and cross-validation silently falls back to unstratified folds. Two of our custom classifiers had been scored **unstratified** while every scikit-learn model got stratified — an unfair comparison in the published table, worst on imbalanced data. Reversing the base classes moved one up on five datasets ($0.688 \rightarrow 0.779$, $0.806 \rightarrow 0.855$) and the other from 0.662 to 0.792 . They had been *underrated* — and one was already being called a consistent winner.

THE PATTERN

These are four unrelated classifier bugs.

They are one bug wearing four hats: **a library default that is wrong for wide, tiny genomics data**. PCA's component count, the RBF kernel width, the Naive Bayes smoothing floor, boosting's minimum leaf size. My fixes are adaptive wrappers keyed to the actual data shape, not tuned constants — a leaf size of 10 scores 0.970 on one small dataset, but choosing it for that reason would just be overfitting to one dataset.

A speedup, and the bug I caught inside it

The evaluation routine was running cross-validated prediction twice — once for labels, then again for probabilities to get the AUC, refitting every fold model from scratch just to read probabilities off it. That is $2k+1$ fits where $k+1$ will do. Walking the folds myself and asking each fitted model for both outputs took an 18-evaluation run from 29.4s to 18.3s with byte-identical metrics across $12 \text{ setups} \times 9 \text{ metrics}$.

My first attempt was faster still and quietly wrong. I derived the labels from the probabilities with an `argmax` — $1.78\times$, but it changed the pairwise classifier's accuracy ($0.855 \rightarrow 0.839$, $0.779 \rightarrow 0.688$). That model predicts on `p  $\geq$  0.5` while `argmax` breaks a 50/50 tie the other way, and since its votes are a fraction over ten pairs, exact ties are common. My pre-check had tested the threshold and I had implemented the `argmax`: **I verified one thing and shipped another**. The old-versus-new metric diff caught it, and there is now a regression test pinning that accuracy.

Two related decisions. The parallel scheduling heuristic moved out of the UI into the pipeline package, because the app had the speedup and Sulaf's cluster harness did not — the classic two-copies-drift problem, now one implementation imported by both. And the optional time budget drops configurations *predicted* to overrun before they start, rather than killing them against a stopwatch: a wall-clock cutoff would let machine load decide which setups survive, so the same data could pick a different winner on a busy node.

The honesty guard, and the hole I found in my own

The guard started simple: if a model's cross-validated predictions are all one class, its accuracy is just the class balance, so say that instead of showing the percentage. Then I extended it to non-convergence, after a 0.580 slipped past the single-class check.

Then I ran the pipeline on the real cohorts and found the hole. The guard catches *total* collapse. It does not catch near-collapse.

THE 298-PATIENT COHORT · MAJORITY BASELINE 0.772

SETUP	ACCURACY	SENSITIVITY	SPECIFICITY
Top three by AUC	0.78–0.79	0.09 / 0.09 / 0.24	—
Pairwise classifier	0.644	0.60	0.66

The three highest-AUC setups clear the baseline by predicting “no response” for almost everyone. The one setup actually identifying responders scores *below* baseline on accuracy. So I closed it: module 5 now also flags sensitivity or specificity below 0.10, and balanced accuracy at or below chance, all four conditions collapsing into one `unreliable` flag every caller reads. Part IV shows it working across 275 setups at once. The same gap drove the metric decision: ranking the automatic search by balanced accuracy instead of raw accuracy.

I gathered the evidence before changing anything. Across ten datasets the pick changed on four, but on three of those sensitivity was *identical*, making it a free tie-break between equally good models. On the fourth it is the difference between finding 16 and 41 of 68 responders. Cheap where accuracy works, decisive where it does not. Accuracy is kept rather than replaced: the results card shows four readings, each with a plain-language line saying what it counts, because “specificity” tells a non-specialist nothing.

Reading the framework’s source instead of guessing

Moving the manual onto its own page surfaced two bugs that no amount of testing my own code would have found, because both lived in how Streamlit works.

`st.Page.__init__` opens with `ctx = get_script_run_ctx()` and returns early if there is no context — so a page constructed at module level, outside a script run, half-initialises *permanently*, because module-level code runs once per process. Worse: `st.navigation` marks each page runnable while `Page.run()` clears that flag, which makes one module-level object shared across sessions a race between two simultaneous users. Pages are now built per rerun.

The second was mine. A CSS rule setting the navigation font overrode Streamlit’s Material icon font, so the icons rendered as the literal words “science” and “menu_book”. I dropped the icons — the design system says no decoration that is not carrying information — and scoped the rule to exclude the icon span anyway.

Writing the manual, and the three things it got wrong

The user manual is thirteen sections: data preparation, every advanced setting, how to read the numbers, comparing setups, imaging, a troubleshooting table, and the limits. The app renders it from that same file, so there is one text rather than two that drift.

I checked every claim against the code rather than writing from memory, and **three were wrong**. The controls are labelled “Normalize”, “Transform (optional)” and “Prediction method” — not the internal preprocessing / feature-map / model names I would have written down; and two of the imaging feature families are off by default because they measure the scanner rather than the patient. Documentation written from memory would have shipped all three.

Design as an extension of the same argument

I redesigned the interface around one rule: **colour is never decorative**. Teal means the patient is predicted to respond, indigo means they are not, and vermilion is reserved for a result that cannot be trusted. That reservation is why the primary button is teal — if vermilion were the accent, every ordinary control would read as an alarm.

The signature detail follows from the honesty guard: when the guard fires, the accuracy is **struck through in place** rather than hidden. A non-learning model’s accuracy is a real number that means nothing, and voiding it where it stands is more honest than removing it. A prediction within ten points of a coin flip is marked borderline and drops to secondary weight — expressed in words and weight, never a fourth colour, because the palette has exactly three meanings and adding one would dilute all of them.

Measuring the rendered page rather than reading the stylesheet was the whole game here. Streamlit styles headings through a container attribute I was not qualifying, so my entire type scale had *silently never applied* — headings rendering at 44px where I had written 22px. No amount of reading the CSS would have found it. Two of the three worst bugs that session were specificity collisions invisible in the source, including one I introduced myself: qualifying a selector to beat Streamlit accidentally raised it above the “does not respond” modifier, so a patient predicted *not* to respond rendered in the teal that means responds — a wrong-meaning colour in the one component the product exists to produce. I caught it in preview, which is the argument for previewing.

That theme has not stopped. A stray glyph after every heading turned out to be a one-letter selector typo — the DOM emits `stHeaderActionElements` and my CSS hid `stHeadingActionElements`; checked in the live DOM, the old selector matched 0 elements and the correct one matched 12. Folding the help section broke because `[data-testid="stExpander"] summary` is a *descendant* selector, so it also matches nested expanders and ties with a plain class — and a tie goes to whichever rule comes later, which was Streamlit’s. And the sign-in card sat in the top-left of a mostly empty slab because it was inheriting the application shell; the shell now restyles itself for that one screen, measured at 197px above and below, then re-measured with an error line injected to confirm it stays centred as the card grows.

The leak that was already live

Auditing the app before any public launch, I found one real breach rather than a hypothetical one. All three PDF builders wrote to fixed paths under a shared directory and the download button re-read the file on every rerun — and Streamlit serves every visitor from a single process, so that directory is shared even though sessions are not.

I confirmed it rather than assuming it: I rebuilt one session's report, had a second session build over it, and hashed the bytes the first session would be served. They matched the *second* session's report, not the first. Those PDFs carry per-patient predictions, so this was a confidentiality breach, and it affected the deployment as it stood.

The fix hands bytes straight to the download button and holds the prediction report in per-session state, with the shared-path constants deleted rather than left unused so the pattern cannot be picked back up. Two regression tests: two sessions must receive different bytes with nothing written to disk, and the interface source must not reintroduce a shared path.

Where a secret could escape

Having set a local password, I owed the team an answer on whether it could leak. Three routes, each checked rather than assumed: git history (`git log -S` across every ref — never committed, and ignored), the Docker image (`COPY . /app` would have taken it, but `.dockerignore` already excludes the secrets file — **.gitignore would not have saved us**), and the deployment sync, which pushes a git branch, so an ignored file cannot ride along. What remains is plaintext on my own disk, and rotating both secrets before launch was already on the deployment checklist.

Integrating a branch without losing the fixes

The imaging work came from a teammate's branch, and a straight merge conflicted in seven places — all in the two files most rewritten here. Taking their side on any of them would have undone real work: their predict tab still wrote reports to a fixed shared path, which is the cross-session leak from two sections ago. So I integrated rather than merged. Their directory-creating helper is deliberately not ported, and their two tests asserting that folder gets recreated are replaced by one asserting the opposite contract — that nothing is written to a shared path at all.

Three fixes to the imported code, one of which changed the architecture. Pillow was imported but undeclared, working only because Streamlit happens to pull it in. And the imaging loader bypassed the matrix-size guard entirely — images being considerably larger than spreadsheets. Rather than patch that one loader, I moved the guard to the single function every data source converges on, **so a new source cannot arrive unbounded again — which is exactly how this one slipped past.**

Naming the thing correctly

I renamed the product to **Treatment Response Predictor**. “Cancer response prediction” parses as the cancer's response — the word doing the work had no object. It was also narrower than the code: nothing in the pipeline is specific to a cancer type or a drug class, and the tool trains on whatever labelled expression matrix you give it. The word “immunotherapy” now appears nowhere in the repository, and the masthead, sign-in gate, page title, three report titles and both documents were changed together so nothing contradicts anything else.

Imaging: measured, not argued

The team's next question was whether to extend the pipeline from expression matrices to image volumes, and whether 3D needs special handling. Rather than reason about it, I measured: a synthetic sphere and *the same slices in random order* produce **byte-identical** feature vectors — maximum difference $0.00e+00$, 102 features either way.

Aggregating across slices is order-independent, so the third dimension is discarded entirely; the pipeline cannot distinguish a coherent 3D shape from shuffled noise, and a 100-slice volume currently yields the same feature count as a single photograph. There is no detection logic to write — the frame count *is* the answer, and the only real gap is file-format readers.

I logged it as an experiment rather than a decision, with the risk stated: at $n = 27-298$, adding ~100 volumetric features is a genuine overfitting hazard, not a free upgrade — particularly since the largest single gain I have measured on any cohort came from *one* clinical column, not a richer feature space.

Building a synthetic image generator to exercise that path closed a loop back to my very first log entry. At an effect size of 1.0 the generated scans separate almost perfectly — 0.95 in 2-D and 1.000 in 3-D — which makes them a useless test bed, because every model ties and nothing demonstrates the ranking or the reliability flags. Default set to 0.5, landing around 0.7–0.8. Two and a half months and an entire second codebase later, the same calibration, for the same reason.

Measuring what we shipped

10 – 17 AUGUST 2026

The product became the instrument: full grids on a real cohort, an imaging comparison, and a cross-cohort test that made me correct my own statistics before it made me correct anything else.

Shipping a real cohort, and a claim I had already got wrong

I checked the licence rather than recalling it — extracted the licence file from inside the source tarball and read it: **CC BY 3.0 Unported**, plain attribution, no non-commercial or share-alike or no-derivatives clause anywhere. Redistributable, with attribution now stated at the point of loading rather than only in a file.

Checking also turned up that **the app shipped no data at all**, while the manual I had written a few days earlier said it did, and so did the picker tooltip. My error, from writing that section before the thing it described existed. I fixed the copy first, then made it true.

Two decisions in the bundling are worth stating because both refuse a convenience. Gene identifiers were relabelled to symbols, 90.7% mapping cleanly; the remaining 9% keep a prefixed identifier rather than being dropped, since losing a tenth of a cohort's genes to get a tidier file is changing the science to suit the packaging. And the mutation-burden option *drops* the 64 patients who lack a value rather than imputing one — filling 21% of a column with a guess would invent the very signal the option exists to measure. I verified the relabelling touched no numbers: features, labels and burden all byte-identical to the source.

The picker also stopped calling the old tumour-versus-normal sets “the benchmark”. They are a benchmark for a different question than the one this tool asks.

The first full grid on real patients

Five preprocessors × five feature maps × eleven models = 275 setups, on 298 patients × 26,128 genes, on the cluster. All 275 completed. Best trustworthy balanced accuracy 0.657, from the rank-based pairwise classifier on raw counts.

The accuracy trap, no longer hypothetical. The highest raw accuracy in the grid is logistic regression at 0.795 — with sensitivity 0.294. Gradient boosting: 0.779 accuracy, sensitivity 0.147. Ranked by accuracy, those win while finding one responder in seven. This is the argument for the balanced-score default, made on the cohort the tool is actually for.

SETUP	ACCURACY	SENSITIVITY	SPECIFICITY
Random Forest — flagged	0.782	0.044	1.000
Logistic Regression — flagged	0.795	0.294	—
Pairwise classifier — kept	0.644	0.735	0.578

112 of 275 setups were flagged — 41% — and 104 of those predict essentially one class. The most seductive catch is that random forest: accuracy 0.782, specificity 1.000, sensitivity 0.044. It calls almost everyone a non-responder and a 23% class split rewards it. Unflagged, it reads as a good model. That is the near-collapse guard from Part III doing exactly the job it was built for, at scale.

And a finding that has driven everything since: **the models rank far better than they classify**. Best AUC in the grid is 0.727, from a setup whose balanced accuracy is 0.541. The ordering is real; the 0.5 cutoff throws it away at 23% prevalence.

Three cluster lessons, each costing a dead run. The sharpest: the parallel backend sizes every worker's numerical thread pool from the *node*'s core count rather than the allocation, so four workers ran at ~400% each against four granted processors and the scheduler killed the job for oversubscription, not memory. **The app has the same hole** — it caps worker count and nothing else.

One clinical column against 26,128 genes

Three arms on the *same* 234 patients, because the burden option drops 64 and a naive before-and-after would confound the covariate with the cohort.

ARM	BALANCED	AUC	SETUPS TRIED
Mutation burden alone (one column)	0.662	0.720	11
Genes + burden (26,129)	0.655	0.714	275
Genes alone (26,128)	0.608	0.682	275

Zero of 288 trustworthy gene-based setups beat the single column. It is not that the maximum edges it — the medians are 0.534 and 0.542 against 0.662, so the entire distribution sits below one clinical number. And the comparison is conservative: the gene arms got 25× more chances at a best-of maximum and still none cleared it.

Honest precision, because the headline number does not deserve more than it can carry: with 61 responders the standard error on balanced accuracy is about 0.036, so ± 0.07 at 95%. Any single pairwise gap here is inside noise. The claim rests on the *0 of 288* pattern, not on the 0.662.

One detail I enjoyed: the winning single-column model is a nearest-shrunk-centroid fit on one feature — which is, in effect, a threshold on mutation burden. A general-purpose search rediscovered the classical high-versus-low biomarker on its own. This is the Part II finding again, now on the merged product with a matched cohort behind it.

Threshold tuning, measured and rejected

If the models rank better than they classify, does choosing the cutoff recover the difference? Fifteen best-AUC setups, cutoff chosen by an inner fold on training data only.

No. Mean gain -0.028 , five of fifteen improved, and the ceiling *fell*: best tuned 0.650 against best default 0.690 . But the pattern is mechanism rather than noise — tuning rescues models whose default cutoff is meaningless at this prevalence, and damages models already sitting at a sensible operating point, which lost between 0.08 and 0.15 . With 61 responders the inner curve is noisy and its argmax overfits, so a good default beats a badly estimated choice.

HALF RIGHT

Ranking ability is being discarded at the cutoff, so tuning the cutoff is free accuracy.

The diagnosis held; the conclusion did not. **The finding worth keeping is about the search, not the threshold:** ranking by balanced accuracy at a default cutoff discards the best-ranking model in the entire experiment as “near chance”. That is a selection blind spot, and it is real whatever we do about thresholds.

The tile cohort I built wrong

800 tiles of real human tissue through the real ingestion route: 0.991 balanced accuracy, median 0.956 across every trustworthy setup. **Which is a warning, not an achievement**, for two reasons, and the second is mine.

First, the distinction is an easy visual call, so a high score says nothing clinical. Second, **the cohort leaked, because I built it wrong**. I flattened 800 tiles into 800 fake patients, so tiles cut from the same slide sat on both sides of the split. The app is not at fault: it expects images grouped per patient and aggregates each patient into one row, which prevents exactly this. The fix is the folder layout, not the cross-validation.

And it cannot be fixed on this dataset — the filenames are random eight-character codes, unique per tile, carrying no slide identity, so the tiles cannot be regrouped. The 0.991 is permanently a tile-level number with slides straddling the split, and it is logged as one.

A free correctness check fell out of the same run: the histogram-based booster swept the top ten with byte-identical metrics across all five preprocessors — correct, since a tree ensemble is invariant to monotone rescaling, and evidence that the assembler is wiring stages the way it claims to.

3-D against 2-D, on the same patients

Two cohorts differing in one thing only: the full volume against a single mid-slice of that same volume. Same 38 patients, same label, same folds, one row per patient, no leakage possible.

38 PATIENTS · 275 MATCHED SETUPS EACH

VIEW	BEST BALANCED	MEDIAN	BEST AUC	MEASURE TIME
Full 3-D volume	0.667	0.530	0.661	1,063 s
2-D mid-slice	0.607	0.530	0.711	6 s

3-D is not worth 177× the compute on this evidence. The +0.060 headline gap sits against a standard error of 0.084. The medians are identical, and at chance. The threshold-free metric points the other way. Paired across all 275 matched setups, 3-D is better in 141 and worse in 125.

Among the twelve setups trustworthy in *both* views, 3-D wins ten, with a rank test at $p = .015$ — and I am not selling that number. Those twelve were selected post-hoc by surviving the guard twice, and 275 setups on 38 patients are heavily correlated, so the test’s independence assumption is violated and the p is optimistic.

What it does show is a small, consistent, negligible gain in exactly the direction the byte-identical experiment in Part III predicted: aggregating slices is order-independent, so depth buys *stability*, not a new kind of information. It does not buy accuracy worth eighteen minutes a cohort against six seconds.

The luck line

Training on one cohort and testing on the other, both directions, 1,100 runs. The apparent result was encouraging: 0.692 balanced, *higher* than anything the same pipeline achieves within the larger cohort. Which is the kind of result that should make you check the null before you check anything else.

CORRECTED · MY OWN NULL

Compare the best of many setups against a null of coin-flip predictors.

Real setups are not coin flips. **40–60% of them predict a single class for everyone**, scoring exactly 0.5 with zero variance — they can never win by luck. Counting them as lottery tickets inflated the ceiling by about 0.05. **The right null holds each setup’s actual prediction pattern and permutes the labels**, which makes the true positives hypergeometric — computable exactly from confusion matrices already stored, in milliseconds, with no refitting.

DIRECTION · SCALING	LIVE SETUPS	CEILING	95TH	OBSERVED
large → small · raw	112/275	0.708	0.783	0.658
large → small · z-scored	169/275	0.725	0.792	0.692
small → large · raw	151/257	0.570	0.598	0.580
small → large · z-scored	271/275	0.591	0.617	0.591

SUPERSEDED

There is zero transfer between these cohorts, in either direction.

One cell sits just above typical luck, though well below the 95th percentile. Nothing anywhere approaches what a signal would need — but **the stronger claim was more than the data supports**, which is its own kind of error.

Two caveats on the ceiling itself, since it is my construction and not a standard: it assumes the setups are independent when taking their maximum, and they are not — shared preprocessing, shared data, often identical predictions — so the true ceiling is lower and this estimate is harsher than reality. And it permutes test labels only; a complete null would refit on shuffled training labels at M times the compute.

The methodological finding is the larger one, and it is about the app. Auto reports the best of 275 setups. At $n = 27$ that maximum reaches 0.77 by chance alone; at $n = 298$, 0.60. The README already warns that the winner's score is optimistic — this quantifies the warning and shows that on a small cohort it is not a formality, it is most of the signal. The app should print the chance ceiling beside the winner. No other honesty feature we have would do as much work.

One more tell: the setups that transferred best were *not* the ones that scored best within their training cohort. That is the signature of noise rather than of a weak but real signal.

The measurement that was right on the wrong data

The SVMs looked broken on the real cohort — warnings everywhere, and zero positives predicted in the threshold experiment. They were not broken. The decision rule was.

Because the plain estimator's probability mode is deprecated, the SVMs had been wrapped to keep reporting a calibrated probability. That swap silently changed a second thing: the plain estimator decides the class from its decision function, while the wrapper takes the argmax of the calibrated probability. Calibration pulls every probability toward the base rate, so at 23% prevalence almost nothing clears 0.5.

DECISION RULE	POSITIVES PREDICTED	BALANCED ACCURACY
Plain estimator	23	0.555
Wrapped, as shipped	6	0.516

SUPERSEDED

The wrapper changes no predictions — measured, 0 of 50 differ.

That measurement was taken on the tumour-versus-normal sets: balanced data, where the swap genuinely is harmless. On an imbalanced cohort — **the only kind this project targets** — it is not. The measurement was correct and the inference from it was wrong, which is the same mistake as choosing a default on synthetic data in Part I, three months apart.

Re-running everything the fix could have touched

Changing how the SVMs decide a class made every artifact produced before it suspect — and one conclusion in particular looked like it might have been me measuring my own defect. So I re-ran both experiments. About an hour of compute.

The three-arm result came back **unchanged to three decimals**. The headline stands, and the fix could not have touched it, because those models were never the wrapped ones. The threshold verdict is identical. But the “tuning rescues broken cutoffs” finding **was inflated by the bug rather than created by it**: a real +0.084 where I had credited +0.143, roughly 40% smaller, and still not worth shipping as a default.

So the bug never affected a single number I had reported. That is worth an hour: the alternative is finding out later that a headline was an artifact of my own wiring.

The revision ledger

Nine conclusions this project reached, then had to withdraw. Each was overturned by a measurement rather than an argument — and each is left standing, struck through, beside what replaced it. This is the through-line

of everything above.

- 1 ~~*Gradient Boosting is the right default model.*~~
→ SVM. The default had been chosen on synthetic data built to suit boosting.
8 real datasets × 5 seeds · 0.9045 vs 0.8368

- 2 ~~*SVM has the best validation AUC; Random Forest has dangerously low sensitivity.*~~
→ Both were single-seed outliers. Three named claims withdrawn; two survived.
seeds 41–45 · sensitivity range 0.526–0.905

- 3 ~~*Neither test cohort carries any usable signal.*~~
→ Wrong twice over: I was under-preprocessing, and my method was wrong for the data.
log2 lifts 0.525 → 0.631 · pairwise method 0.726

- 4 ~~*Literal IMPRES is a strong, significant predictor.*~~
→ In-sample optimism. On an independent cohort it is the worst method in the table.
0.825 / 0.752 → 0.560, p = 0.078 (n.s.)

- 5 ~~*Biology-driven methods beat generic machine learning.*~~
→ An artifact of small n. The learner that was worst at n ≈ 30 is best at n = 298.
SVM + log2 · 0.697, p = 0.020

- 6 ~~*0.991 balanced accuracy on real human tissue.*~~
→ I built the cohort wrong — tiles from one slide sat on both sides of the split.
800 tiles flattened into 800 fake patients

- 7 ~~*Ranking is discarded at the cutoff, so tuning the cutoff is free accuracy.*~~
→ It lowers the ceiling and hurts on average; it only rescues models whose default was meaningless.
mean -0.028 · best tuned 0.650 vs 0.690 default

- 8 ~~*Zero transfer between the cohorts, in either direction.*~~
→ Too strong. One cell sits just above typical luck — and the null I used to say it was wrong.
corrected null · 1,100 runs · ceiling was inflated ~0.05

- 9 ~~*The calibration wrapper changes no predictions — measured, 0 of 50.*~~
→ True on balanced data. False on the imbalanced cohorts this project exists for.
23 positives → 6 · balanced 0.555 → 0.516

What I would defend

Measure the thing instead of reasoning about it.

Every real bug in this project — the kernel width, the base-class ordering, the 3.4-hour configuration, the 2D-versus-3D question, the type scale that never applied — came from running it. None came from code review or from the test suite alone.

Test a rejection as carefully as an adoption.

Before declining to port the curated gene signatures, I verified the gene mapping worked completely — 18/18, 8/8, 2/2 and 7/7 genes matched — *then* measured them as features: best balanced accuracy 0.539 against 0.589 for the full transcriptome. A negative result you can defend is worth more than a feature you cannot.

Check that a fix generalizes before believing it.

The kernel-width fix was not finished after one dataset; it was finished after eight. And when a fix lands late, re-run what it could have touched: an hour of compute confirmed six reported numbers were safe, and caught one that had been inflated by 40%.

Argue against your own best result.

IMPRES at 0.825 was the strongest number I had produced, and writing down why it was probably circular is what made the decisive experiment obvious.

Leave the retraction visible.

A log that quietly replaces yesterday's conclusion is much less useful than one that shows what changed and why — for the next person, and for me three weeks later.

Open, and honest about it

The one worth doing next is showing the chance ceiling beside Auto's winner: *comparing M setups on N patients, random guessing reaches X ; your winner scored Y* . Part IV computes that line in milliseconds from confusion matrices the app already stores — and at small n it is not a footnote on the result, it is most of the result.

Also open: the app caps worker count but not each worker's thread pool, so it carries the same oversubscription hole that got a cluster job killed. Imaging cohorts need their folder layout enforced, since a flat one is what let me leak a split. `class_weight="balanced"` is the next threshold-free attempt at the prevalence problem. Auto's selection bias is disclosed but not corrected, because an honest estimate means nested cross-validation over the whole grid and

nobody has found a way to afford it. Boosting still wants an adaptive minimum leaf size rather than a constant, and the scheduler's cost weights are measurably mis-calibrated. The deployment path is still an open team decision.

And one that is not a code problem at all: accepting real patient uploads on a public URL is a data-governance question, and it needs answering with the professor before launch, not after.

ANGELINA · BOSTON UNIVERSITY SUMMER RESEARCH · LAST UPDATED 17 AUGUST 2026